

Tutorial

TopoScale: An Automated Topological Framework for Robust Multi-Scale Data Analysis

Naseem Zoha Ansari¹, Subhajit Bandyopadhyay², Sudarshan Gogoi¹, Mubashar Rashid³, Amit Chakraborty^{*1}

¹Department of Mathematics, Sikkim University, Gangtok, 737102, Sikkim, India.

²Department of Computer Applications, Sikkim University, Gangtok, 737102, Sikkim, India

³Department of Mathematics, Indian Institute of Technology, Kanpur, India.

- ① How to install Python in Anaconda Navigator
- ② Tutorial for Coordinate points
- ③ Tutorial for Correlation Matrix

① How to install Python in Anaconda Navigator

② Tutorial for Coordinate points

③ Tutorial for Correlation Matrix

Step 1 : Open Anaconda Navigator

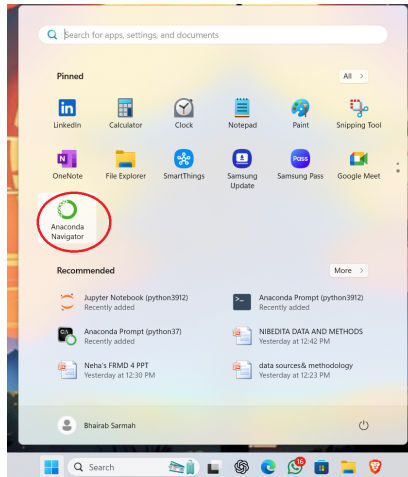


Figure 1: Select the Anaconda Navigator file from the destination or in Strat menu

Step 2 : Selecting Python 3.9.21 for our program use

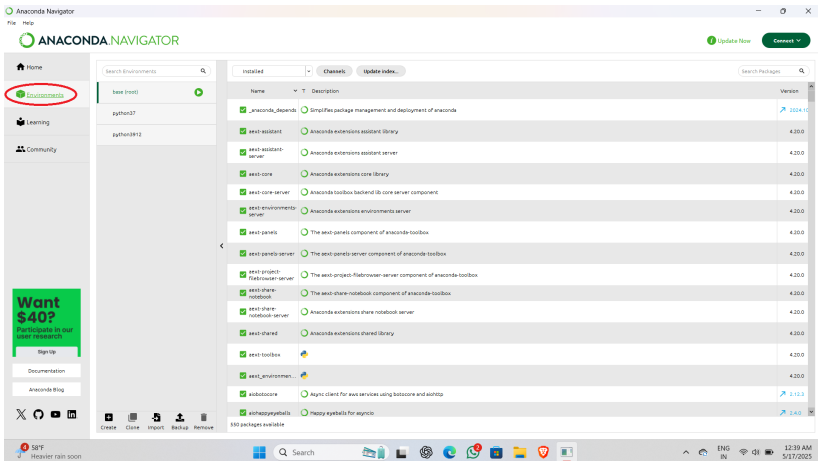


Figure 2: Click on the Environment tab just below Home tab

Step 2 : Selecting Python 3.9.21 for our program use

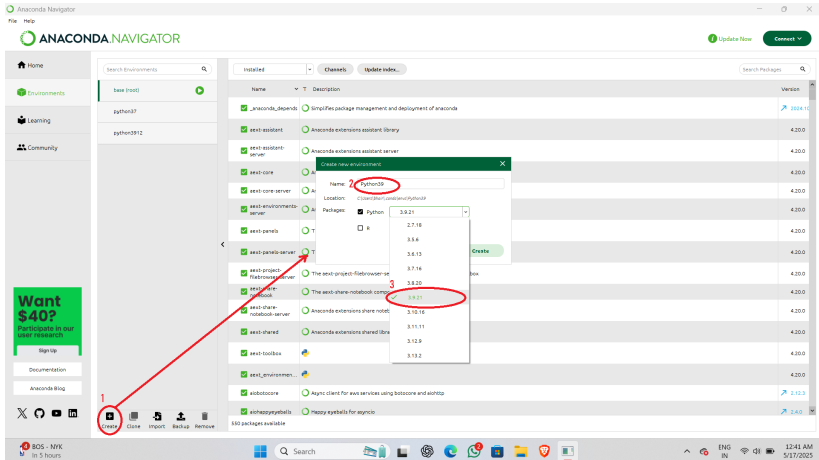


Figure 3: Click Create, name your environment in the dialog box that appears, select Python 3.9.21

Step 2 : Selecting Python 3.9.21 for our program use

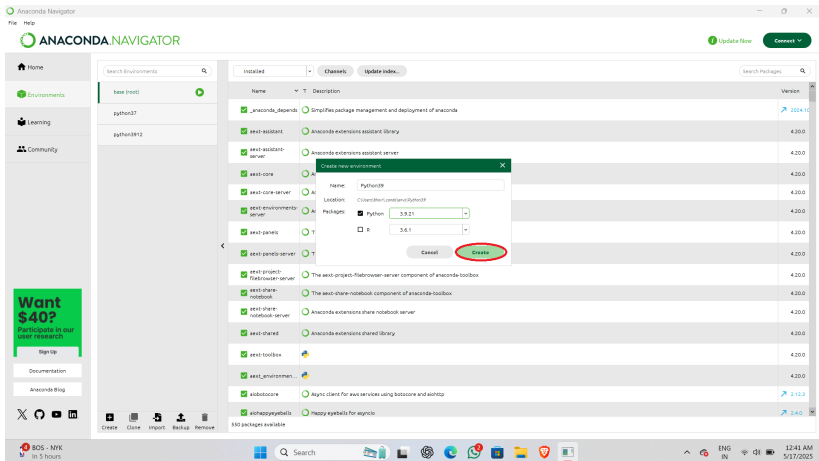


Figure 4: Hit on create button to confirm

Step 2 : Selecting Python 3.9.21 for our program use

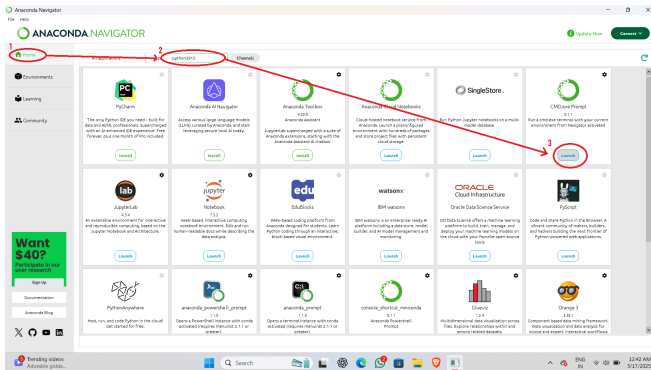


Figure 5: Navigate to the **Home** tab, verify the created environment under the *Applications on* section, and launch **CMD.exe** to install the required packages.

Step 3 : Installation of the required packages

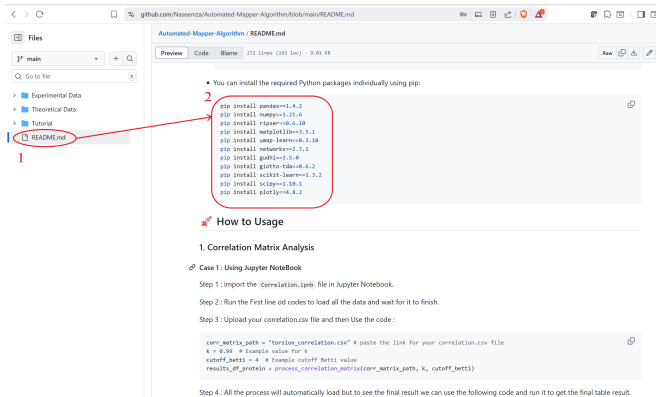


Figure 6: Visit the GitHub repository

<https://github.com/Naseemza/TopoScale>, open the README.md file to identify the required package name, and use the cmd.exe dialog box to install all compatible packages using pip.

Step 3 : Installation of the required packages

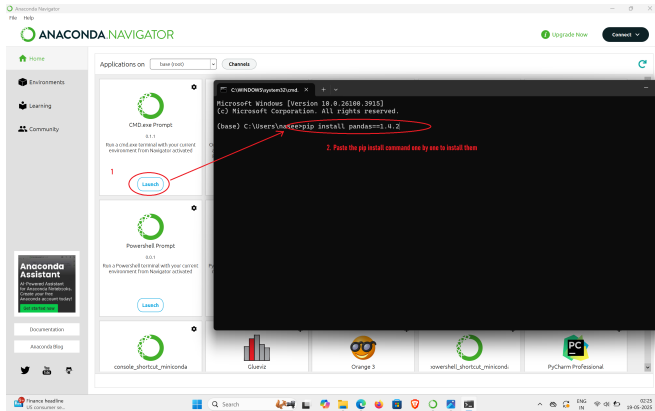


Figure 7: Open Anaconda Navigator, launch the Command Prompt (CMD.exe), and sequentially execute each pip command to install the required packages.

- ① How to install Python in Anaconda Navigator
- ② **Tutorial for Coordinate points**
- ③ Tutorial for Correlation Matrix

Step 1 : Download the Torus.ipynb file

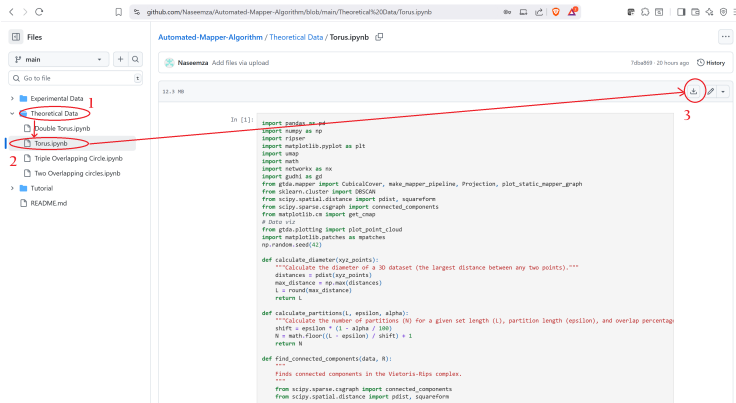


Figure 8: Click on the **Theoretical Data** section, open the **Torus.ipynb** file (or any other), and click **Download** to save it from <https://github.com/Naseemza/TopoScale>.

Step 1 : Download the Torus.ipynb file

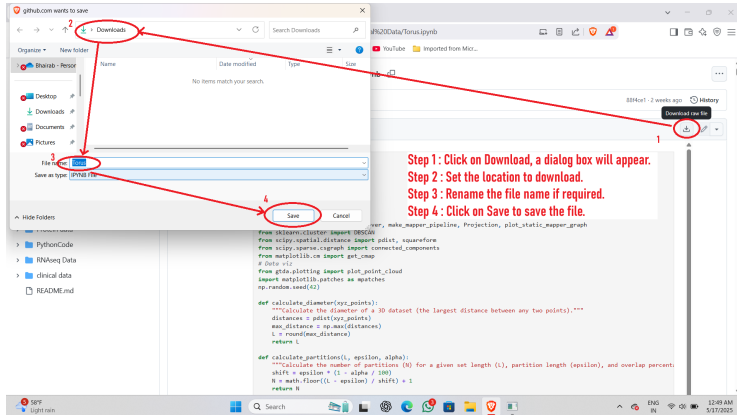


Figure 9: The process for downloading a file: click, locate, rename and save

Step 2 : Uploading the .ipynb file

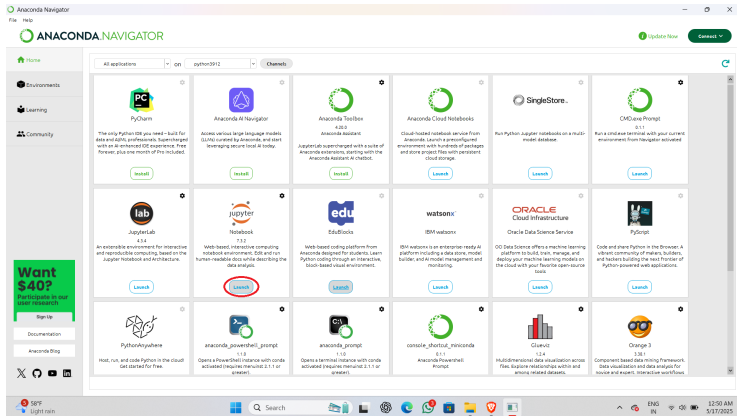


Figure 10: Open Jupyter Notebook and wait for it to load from Anaconda Navigator application

Step 2 : Uploading the .ipynb file

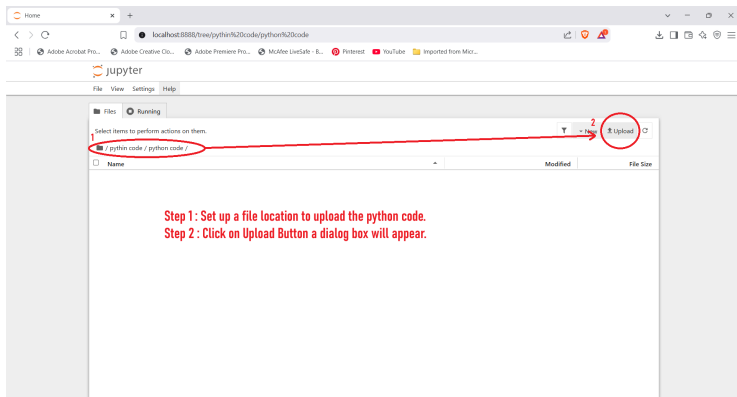


Figure 11: In the **Home** tab of the browser, navigate to the desired location, then click **Upload** to select and upload the previously downloaded file.

Step 3 : Lunch the .ipynb file

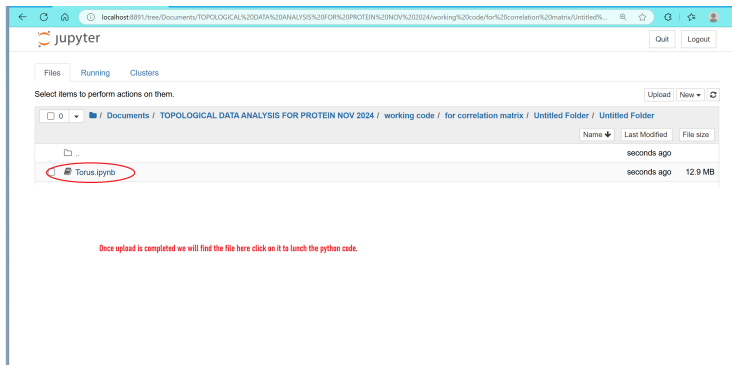


Figure 12: Click on the uploaded file to lunch the python code

Step 4 : Run the python code

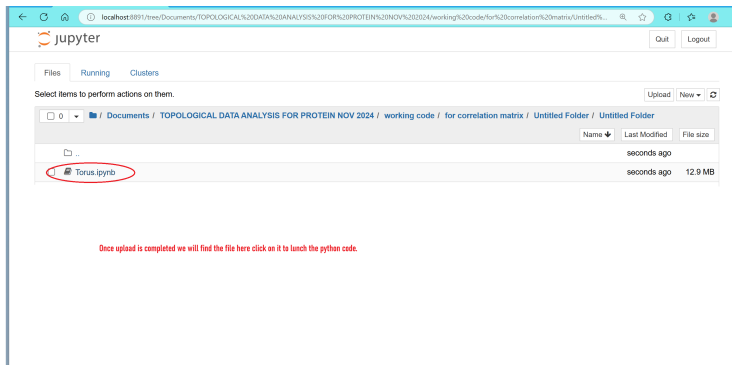


Figure 13: Click on the uploaded file to lunch the python code

Step 4 : Run the python code

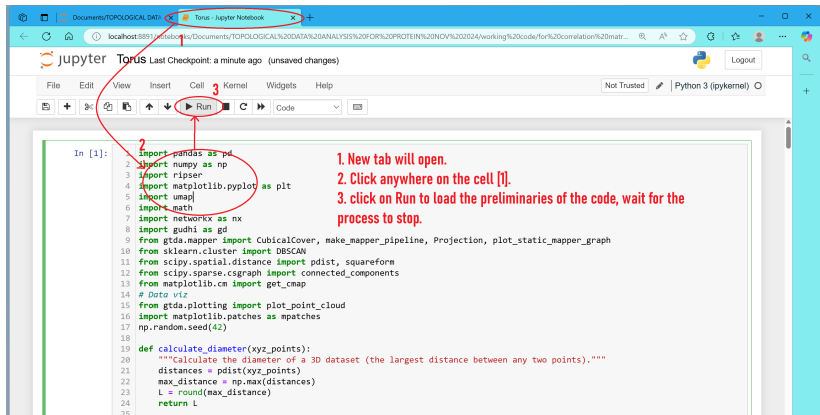
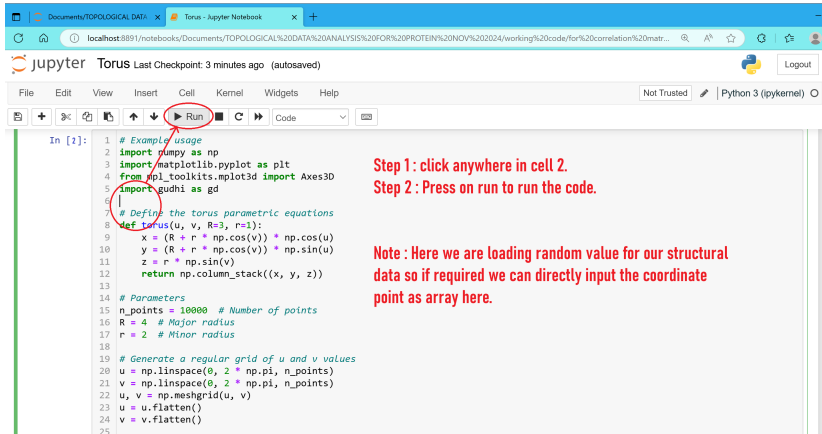


Figure 14: A new tab will open. Click on cell [1], then click Run to load the preliminaries of the code. Wait for the process to complete before proceeding.

Step 4 : Run the python code



The screenshot shows a Jupyter Notebook titled 'Torus - Jupyter Notebook' in a web browser. The notebook contains Python code for generating torus data. A red circle highlights the 'Run' button in the toolbar, and another red circle highlights the 'def torus' function definition in the code cell. Red arrows point from the text instructions to these elements.

```
In [2]: 1 # Example usage
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from mpl_toolkits.mplot3d import Axes3D
5 import gudhi as gd
6
7 # Define the torus parametric equations
8 def torus(u, v, R=3, r=1):
9     x = (R + r * np.cos(v)) * np.cos(u)
10    y = (R + r * np.cos(v)) * np.sin(u)
11    z = r * np.sin(v)
12    return np.column_stack((x, y, z))
13
14 # Parameters
15 n_points = 10000 # Number of points
16 R = 4 # Major radius
17 r = 2 # Minor radius
18
19 # Generate a regular grid of u and v values
20 u = np.linspace(0, 2 * np.pi, n_points)
21 v = np.linspace(0, 2 * np.pi, n_points)
22 u, v = np.meshgrid(u, v)
23 u = u.flatten()
24 v = v.flatten()
25
```

Step 1: click anywhere in cell 2.

Step 2: Press on run to run the code.

Note : Here we are loading random value for our structural data so if required we can directly input the coordinate point as array here.

Figure 15: Follow thw process to load the coordinate data.

Step 4 : Run the python code

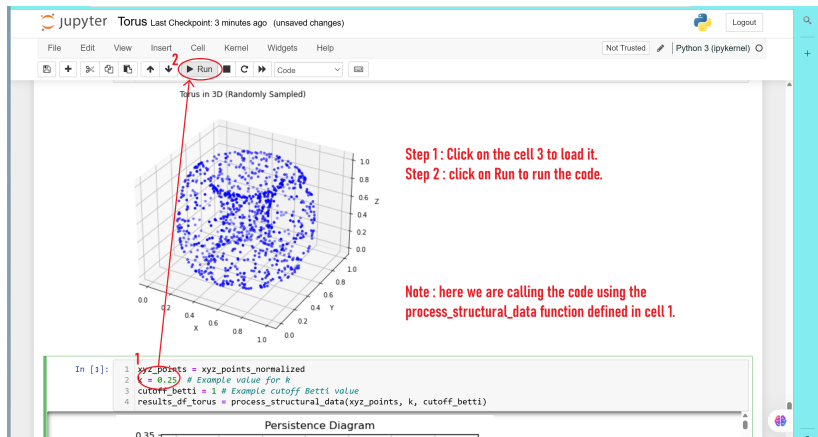


Figure 16: Follow thw process to run the code for your data.

Step 5 : Print thr final result

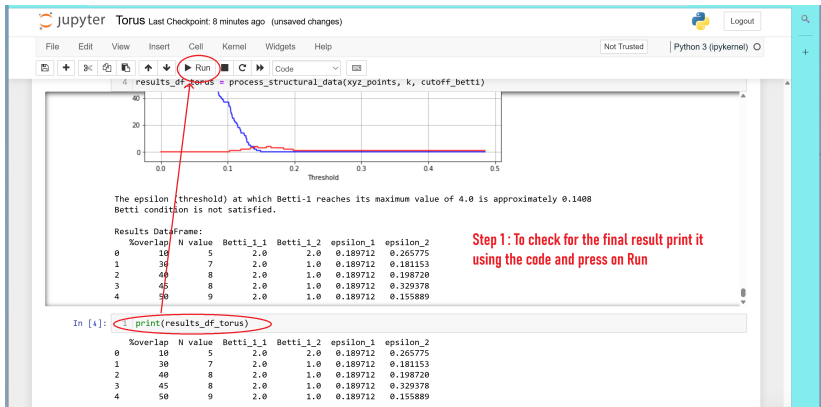


Figure 17: Follow the process to get the final result table.

- 1 How to install Python in Anaconda Navigator
- 2 Tutorial for Coordinate points
- 3 Tutorial for Correlation Matrix**

Step 1 : Uploading the .ipynb and .csv file

The screenshot shows the JupyterLab interface. At the top, the 'jupyter' logo is on the left, and 'Quit' and 'Logout' buttons are on the right. Below the logo, there are tabs for 'Files', 'Running', and 'Clusters'. A message says 'Select items to perform actions on them.' To the right of this message, there is a toolbar with 'Upload', 'New', and a refresh icon. The 'Upload' button is circled in red and labeled with a red '1'. Below the toolbar, the file browser shows the path: 'Documents / TOPOLOGICAL DATA ANALYSIS FOR PROTEIN NOV 2024 / working code / for correlation matrix / Untitled Folder / clinical data / Untitled Folder'. There are three files listed: a folder icon, 'clinical_data.ipynb', and 'clinical_corr_2.csv'. The 'clinical_data.ipynb' file is circled in red and labeled with a red '2'. The 'clinical_corr_2.csv' file is circled in red and labeled with a red '3'. The table has columns for 'Name', 'Last Modified', and 'File size'.

Name	Last Modified	File size
.	seconds ago	
clinical_data.ipynb	a month ago	8.73 MB
clinical_corr_2.csv	4 months ago	18.8 kB

1. Upload the clinical_data.ipynb file along with the correlation matrix .csv as done for coordinate points.
2. Click on the .ipynb file to run it and wait for new tab to open.

Figure 18: We follow the same steps of downloading the files and then similarly as coordinate files we upload the files here.

Step 2 : Run the code

The screenshot shows the JupyterLab interface with a code cell containing Python code. Red annotations highlight the 'Run' button in the toolbar and the code cell itself. The code is as follows:

```
In [1]: 1 import pandas as pd
2 import numpy as np
3 import ripser
4 import matplotlib.pyplot as plt
5 import unap
6 import math
7 import networkx as nx
8 import gudhi as gd
9 from gtda.mapper import CubicalCover, make_mapper_pipeline, Projection, plot_static_mapper_graph
10 from sklearn.cluster import DBSCAN
11 from scipy.spatial.distance import pdist, squareform
12 from scipy.sparse.csgraph import connected_components
13 from matplotlib.cm import get_cmap
14 # Data viz
15 from gtda.plotting import plot_point_cloud
16 import matplotlib.patches as mpatches
17 np.random.seed(42)
18
19 def calculate_diameter(xyz_points):
20     """Calculate the diameter of a 3D dataset (the largest distance between any two points)."""
21     distances = pdist(xyz_points)
22     max_distance = np.max(distances)
23     L = round(max_distance)
24     return L
25
26 def calculate_partitions(L, epsilon, alpha):
27     """Calculate the number of partitions (N) for a given set length (L), partition length (epsilon), and overlap percentage
28     shift = epsilon * (1 - alpha / 100)
29     N = math.floor((L - epsilon) / shift) + 1
30     return N
31
```

Step 1: Click on cell 1 and the press Run.

Note: Wait for the process to finish.

Figure 19: We run the code step by step

Step 3 : Print the final result

The screenshot shows a Jupyter Notebook titled 'clinical data' with the following code in cell [3]:

```

1 # Example usage
2 corr_matrix_path = "D:\\correlation matrix sudarsan\\CorrMatrix\\ClinicalBreastData\\correlation_matrix.csv"
3 k = 0.04 # Example value for k
4 cutoff_betti = 0.5 # Example cutoff Betti value
5 results_df_clinical = process_correlation_matrix(corr_matrix_path, k, cutoff_betti)

```

Below the code, a warning message is displayed: "C:\Users\naseef\anaconda3\lib\site-packages\ripriser\ripriser.py:247: UserWarning: The input matrix is square, but the distance matrix flag is off. Did you mean to indicate that this was a distance matrix? warnings.warn()".

A persistence diagram plot is shown, titled 'Persistence Diagram'. The x-axis is labeled 'dim' and ranges from 0.0 to 1.4. The y-axis is labeled 'Death' and ranges from 0.9 to 1.4. The plot shows two clusters of points: one at the bottom left (blue dots) and one at the top right (orange dots). A legend in the bottom right corner indicates 'H0' for blue and 'H1' for orange.

Below the plot, cell [4] contains the code:

```

1 print(results_df_clinical)

```

The output of the code is a table with the following data:

	Koverlap	N value	Betti_1_1	Betti_1_2	epsilon_1	epsilon_2
0	50	29	2.0	2.0	1.306469	1.399681
1	55	32	2.0	2.0	1.306469	0.576101

Red annotations in the image include:

- A red circle around the 'Run' button in the Jupyter interface.
- A red circle around the file path in the code.
- A red circle around the 'print(results_df_clinical)' line in cell [4].
- Red numbers 1, 2, and 3 indicating steps: 1. Check for the correct path of the correlation matrix and the run the code using the code and run. 2. Wait for the process to end. 3. to get the main result we can print the result using the code.

Figure 20: Follow the process to get the final result table. We need to double check for the .csv file and put the path in the section to be read automatically by the code.

Thanks For Your Attention!